

Entity Framework Core – ASP.NET Essentials



1. Introduction to CinemaWebApp and Entity Framework Core

Welcome to the Entity Framework Core Workshop! In this workshop, we will build the CinemaWebApp step by step, focusing on backend development using ASP.NET Core and Entity Framework Core. Throughout this journey, you will learn how to design and manage a database, work with models, and implement CRUD operations.

Frontend development will be covered in a future ASP.NET course, so for now, we will focus exclusively on the database and backend logic

Start Thinking About Your Graduation Project

As we progress with building the CinemaApp, it's important to highlight that this project closely resembles what you will need to create and defend for your graduation:

- The concepts and architecture we use here can be adapted to your own graduation project
- Think about how you will design your own database and entities for your final project

Start Exploring ASP.NET Architecture

You Will Need This for Your Graduation Project

- Just like with the database design, your graduation project will require you to structure your backend, business logic, and frontend properly
- The sooner you start exploring ASP.NET architecture, the easier it will be when you need to build your own project

Workshop Goals

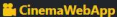
- Create an empty ASP.NET Core MVC project and set up EF Core
- Define and configure database models using EF Core
- Apply migrations to generate the database schema
- Seed initial data into the database
- Implement basic CRUD operations using EF Core

2. Demonstration of the Application

Before starting the development, let's explore the final outcome of the CinemaWebApp. This will help you understand the goal and structure of the application you will be building:

- These are the credentials that could be used initially for exploring the application

```
"admin@example.com", "Admin@123", "Admin");  
"appManager@example.com", "123asd", "Manager");  
"appUser@example.com", "123asd", "User");
```

 Cinemas Movies Tickets Watchlist

Hello, appUser@example.com! Logout



WELCOME TO CINEMAWEBAPP!

Discover the best cinemas and movies tailored for your entertainment.

[Explore Cinemas](#) [Browse Movies](#)

Popular Choices

**Cinema City Sofia**

Voted as the best cinema experience this month.
Over 10,000 visitors last month.

**Dune: Part Two**

The sci-fi epic that has everyone talking!
Rated ★★★★★ (9.2/10 IMDb)

© 2024 - 2025 CinemaWebApp | Bringing Cinema and Movies to You

Discover Cinemas Near You

← Back to Home Page

All Cities



Cinemax

Varna

[View Program](#)

Explore Movies

← Back to Home Page



The Shawshank Redemption

Genre: Drama | Director: Frank Darabont
Released: September 1994

[View Details](#) [Add to Watchlist](#)



Harry Potter and the Goblet of Fire

Genre: Fantasy | Director: Mike Newel
Released: November 2005

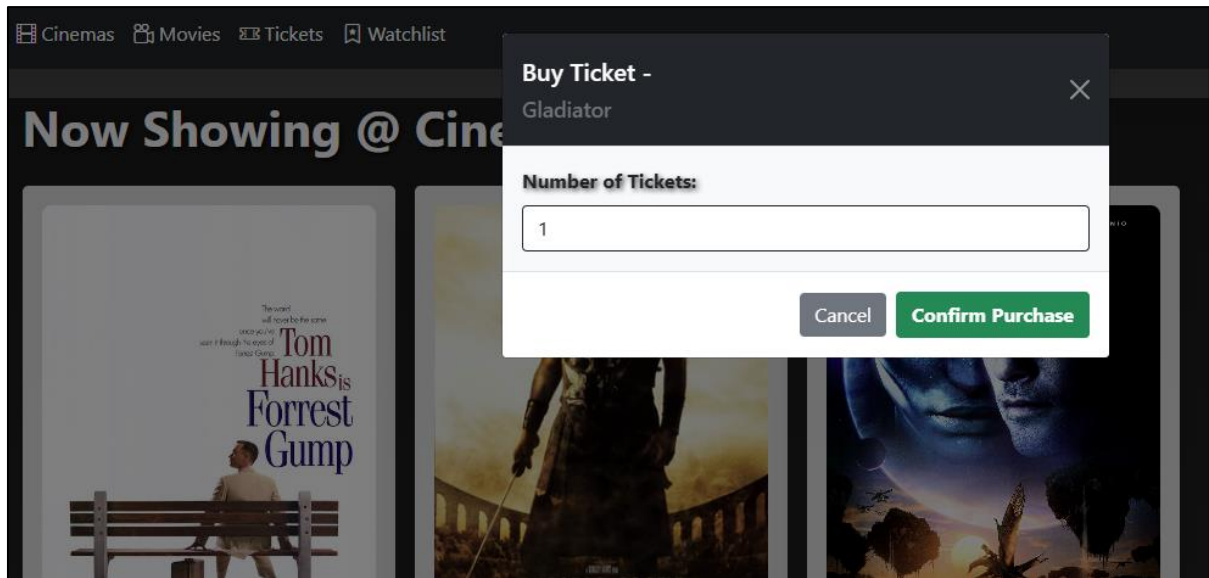
[View Details](#) [Add to Watchlist](#)



Forrest Gump

Genre: Drama | Director: Robert Zemeckis
Released: July 1994

[View Details](#) [Add to Watchlist](#)



My Tickets

[← Back to Home Page](#)



Gladiator
Cinema: Cinemax
Price: **\$0.00**

3 x

My Watchlist

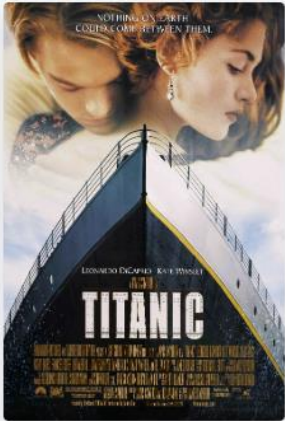
[← Back to Home Page](#)



Titanic
Genre: Romance
Release Date: 12/1997

[View Details](#) [Remove](#)

Titanic



Genre: Romance
Release Date: 1997-12-19
Director: James Cameron
Duration: 195 min
A love story unfolds on the doomed voyage of the Titanic.

[Close](#)

The diagram illustrates the three main management sections of the application, each represented by a colored box with a corresponding icon, title, description, and action button.

- Manage Users** (Yellow box):
 - Icon: Two stylized people.
 - Description: Add, edit, or remove users and assign roles.
 - Action: [Go to Users](#)
- Manage Cinemas** (Blue box):
 - Icon: A film strip.
 - Description: Create, update, and delete cinema entries.
 - Action: [Go to Cinemas](#)
- Manage Movies** (Red box):
 - Icon: A video camera.
 - Description: Add new movies, edit details, or remove them.
 - Action: [Go to Movies](#)

[← Back to Dashboard](#)

Email	Roles	Actions
admin@example.com	Admin	Cannot edit Admin
appManager@example.com	Manager	Manager Assign Remove Delete
appUser@example.com	User	User Assign Remove Delete

[View, edit, or delete cinema entries.](#)

[← Back to Dashboard](#)

[+ Add New Cinema](#)

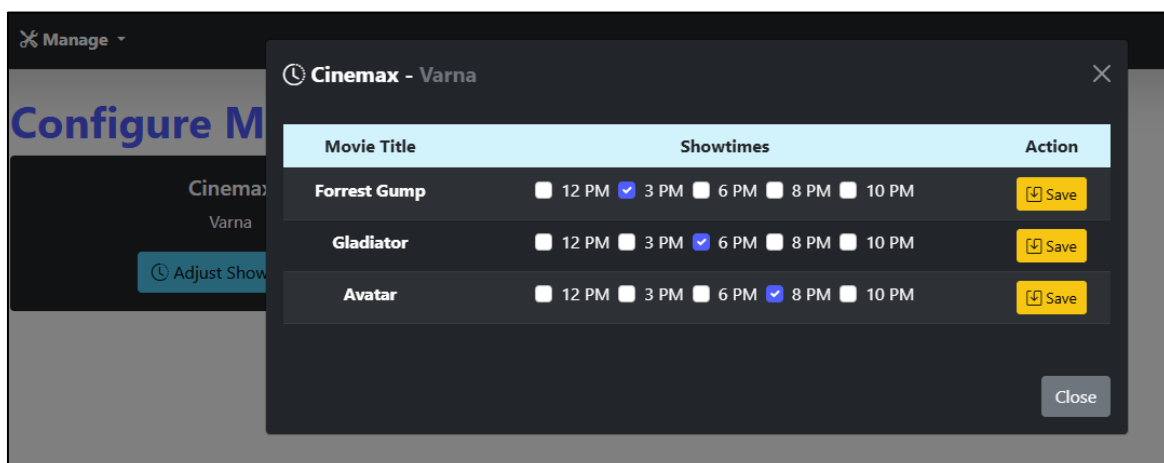
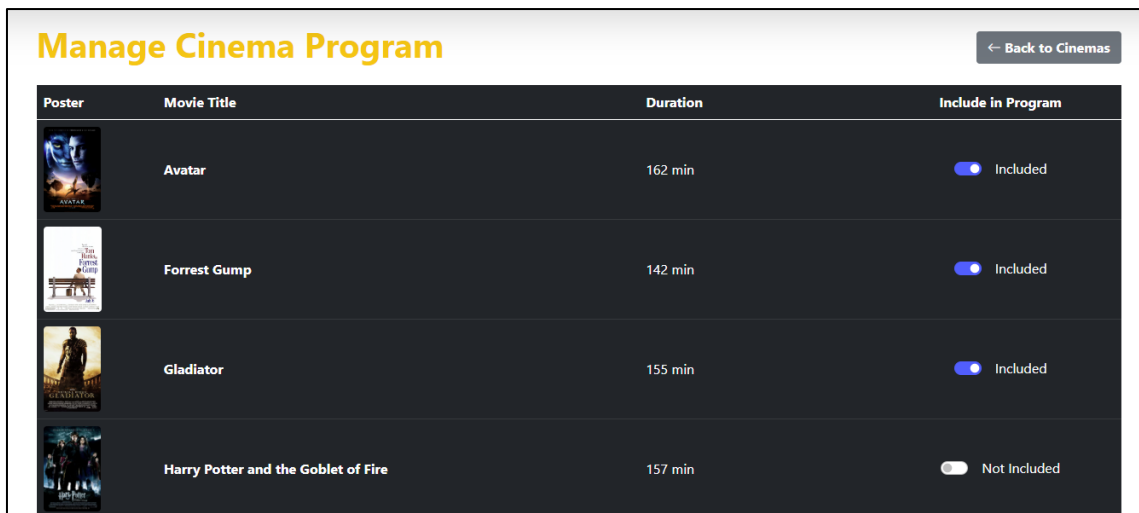
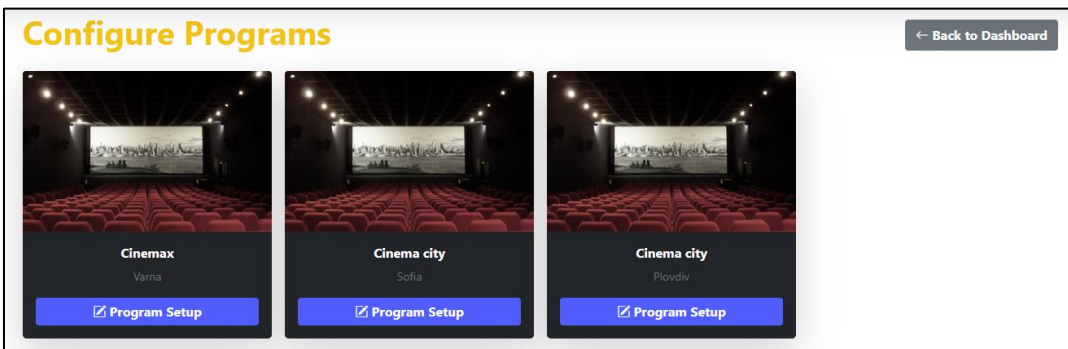
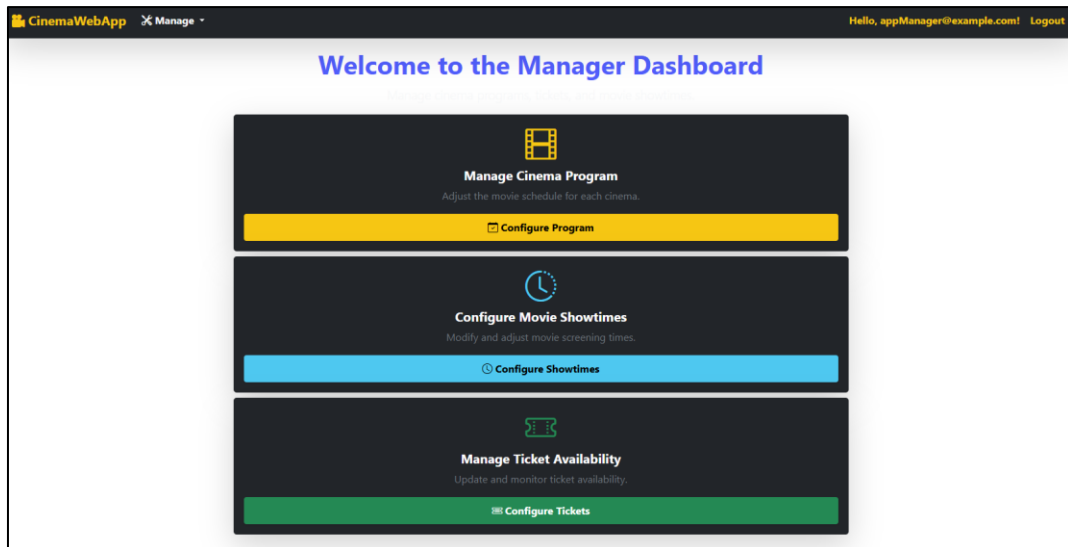
Name	Location	Actions	
Cinema city	Plovdiv	✎ Edit	🗑 Delete
Cinema city	Sofia	✎ Edit	🗑 Delete
Cinemax	Varna	✎ Edit	🗑 Delete

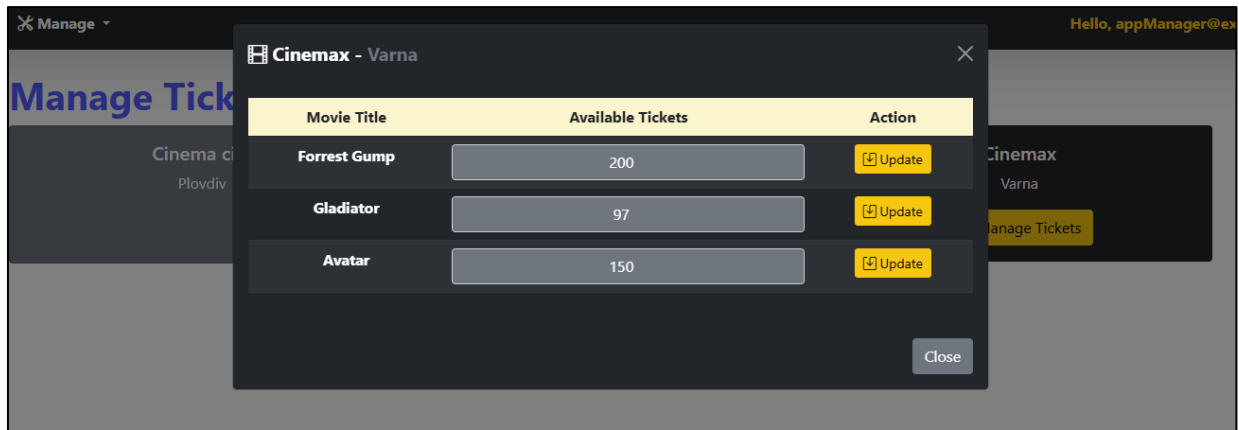
[View, edit, or delete movie entries.](#)

[← Back to Dashboard](#)

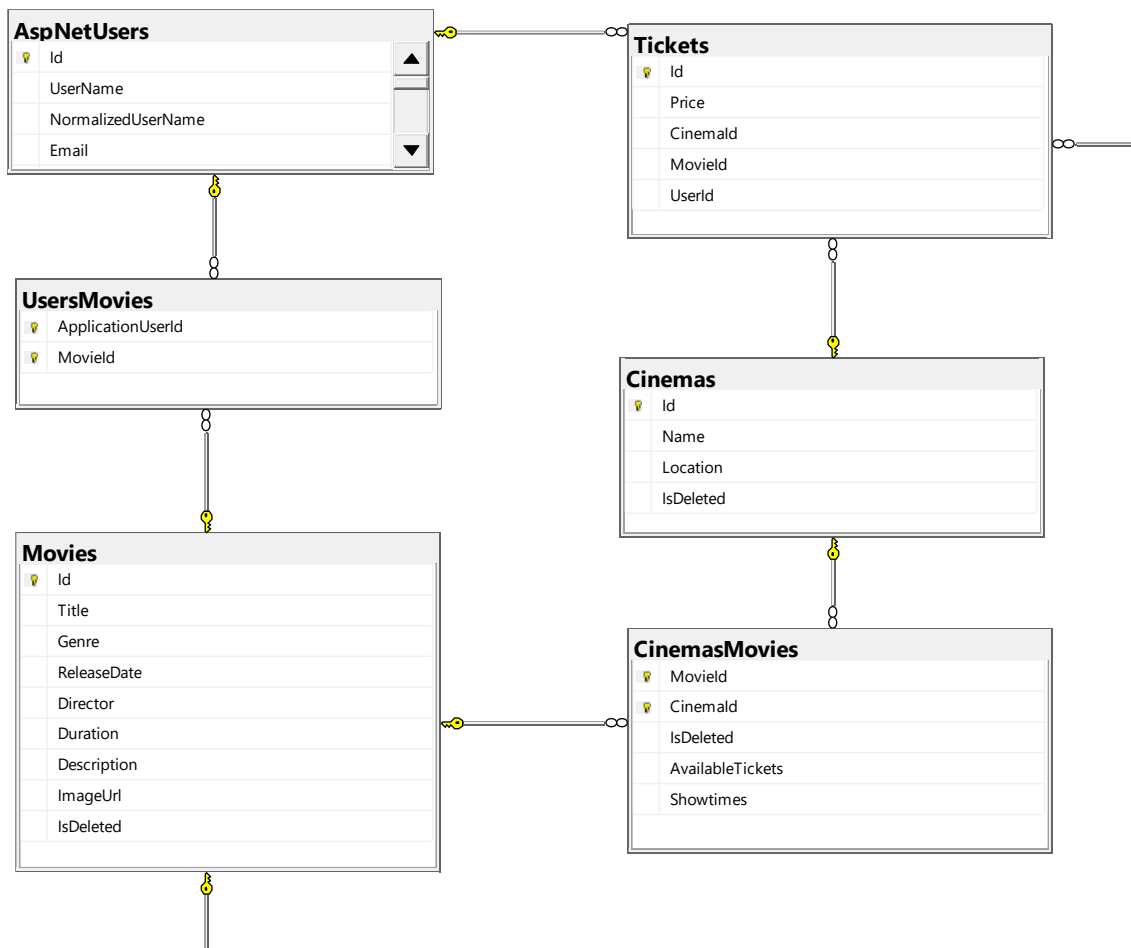
[+ Add New Movie](#)

Title	Genre	Duration	Director	Release Date	Actions	
The Shawshank Redemption	Drama	142 min	Frank Darabont	1994-09-23	✎ Edit	🗑 Delete
Harry Potter and the Goblet of Fire	Fantasy	157 min	Mike Newel	2005-11-01	✎ Edit	🗑 Delete
Forrest Gump	Drama	142 min	Robert Zemeckis	1994-07-06	✎ Edit	🗑 Delete
Titanic	Romance	195 min	James Cameron	1997-12-19	✎ Edit	🗑 Delete
Gladiator	Action	155 min	Ridley Scott	2000-05-05	✎ Edit	🗑 Delete





Database Overview



3. Creating an ASP.NET Core MVC Project

Setting Up the Project

Create a New ASP.NET Core MVC Project:

- Open Visual Studio and select Create a new project
- Choose ASP.NET Core Web App (Model-View-Controller) and click Next
- Name the project (CinemaApp) and choose a location to save it
- Select .NET 8 as the framework and click Create
- Run the application to ensure it is working properly (Ctrl + F5)

Visual Studio 2022



Create a new project

Choose a project template with code scaffolding to get started

Create a new project

Recent project templates

A list of your recently accessed templates will be displayed here.

mvc Clear all

All languages All platforms All project types

ASP.NET Core Web App (Model-View-Controller)
A project template for creating an ASP.NET Core application with example ASP.NET Core **MVC** Views and Controllers. This template can also be used for RESTful HTTP services.
C# Linux macOS Windows Cloud Service Web

ASP.NET Core Web App (Model-View-Controller)
A project template for creating an ASP.NET Core application with example ASP.NET Core **MVC** Views and Controllers. This template can also be used for RESTful HTTP services.
F# Linux macOS Windows Cloud Service Web

ASP.NET Core Web API
A project template for creating an ASP.NET Core application with an example Controller for a RESTful HTTP service. This template can also be used for ASP.NET Core **MVC** Views and Controllers.
F# Linux macOS Windows Cloud Service Web Web API

Configure your new project

ASP.NET Core Web App (Model-View-Controller)

C# Linux macOS Windows Cloud

Project name

CinemaApp

Location

E:\VisualStudio\

Solution name ⓘ

CinemaApp

☐ Place solution and project in the same directory

Project will be created in "E:\VisualStudio\CinemaApp\CinemaApp."

Additional information

ASP.NET Core Web App (Model-View-Controller)

C# Linux macOS Window

Framework ⓘ

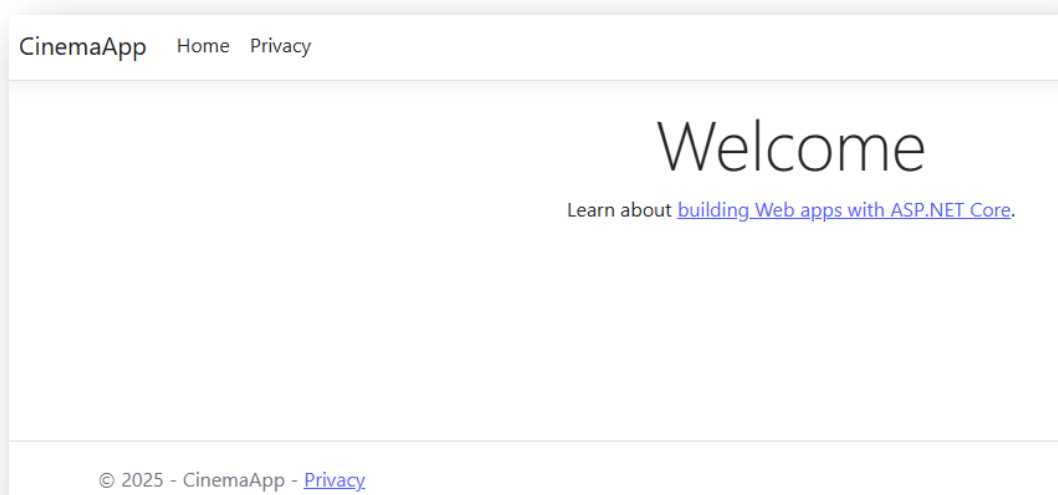
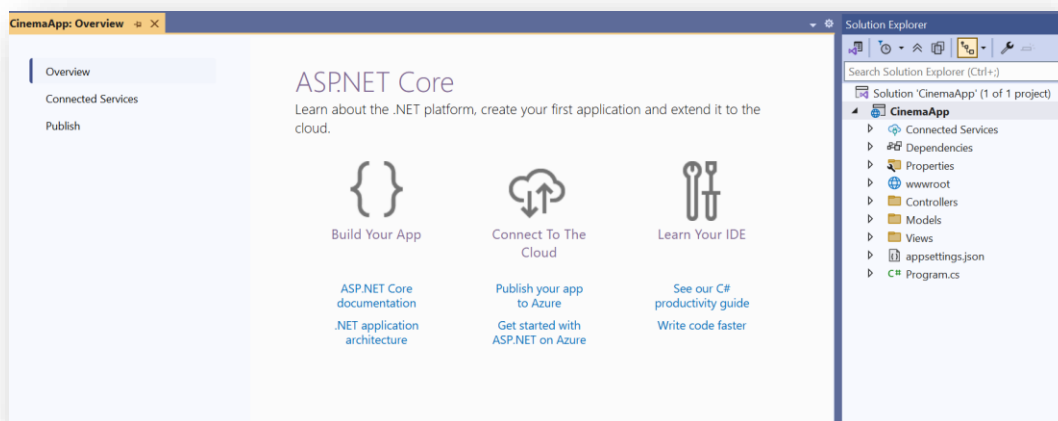
.NET 8.0 (Long Term Support)

Authentication type ⓘ

Individual Accounts

☒ Configure for HTTPS ⓘ

☐ Enable container support ⓘ

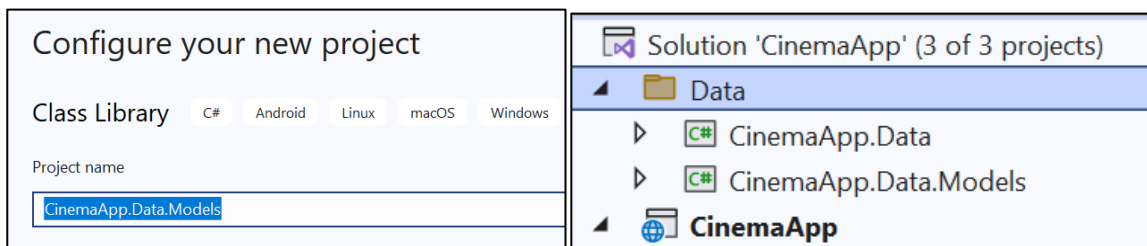
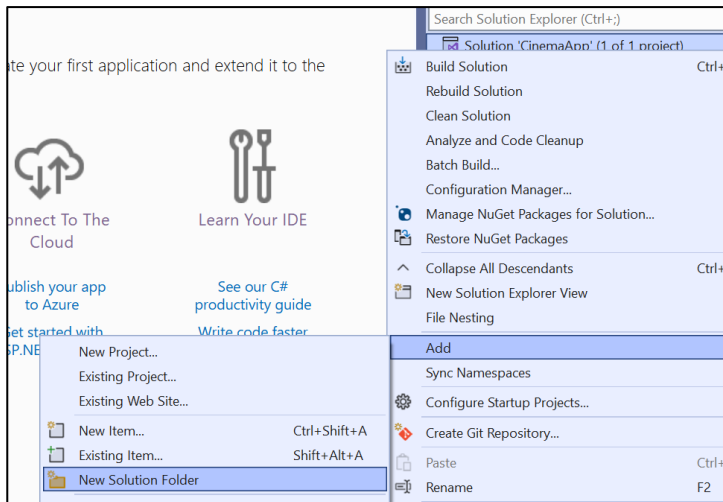


4. Setting Up the Project Architecture

To keep the project **well-organized**, we will create the **Data** folder, which will contain all database-related components

Create the Data Folder

- In **Solution Explorer**, right-click the project and select **Add > New Folder**.
- Name the folder **Data** (for organizational purposes)
- Creating Class Library Projects for Separation of Concerns
 - **CinemaApp.Data.Models** – Stores all database models
 - **CinemaApp.Data** – Handles database access (DbContext, repositories, configurations)
- Adding References
 - **Right-click CinemaApp.Data** → Select **Add Project Reference**
 - Select **CinemaApp.Data.Models** and click **OK**
- Ensuring the package dependencies are correctly installed



5. Install EF Core Packages in CinemaApp.Data

Since we are working with **Entity Framework Core (EF Core)** for database management, we need to install the required packages:

 **Microsoft.EntityFrameworkCore.SqlServer**  by aspnet, dotnetfram
Microsoft SQL Server database provider for Entity Framework Core.

 **Microsoft.EntityFrameworkCore.Tools**  by aspnet, dotnetframewor
Entity Framework Core Tools for the NuGet Package Manager Console in Visual Studio.

 **Microsoft.AspNetCore.Identity.EntityFrameworkCore**  by aspnet
ASP.NET Core Identity provider that uses Entity Framework Core.

 **Microsoft.EntityFrameworkCore.Design**  by aspnet, dotnetframew
Shared design-time components for Entity Framework Core tools.

Once all packages are installed, we can move forward with creating **all data models**

6. Creating Data Models

We will now create the **core entity models** inside **CinemaApp.Data.Models** before implementing the database context

ApplicationUser

Extends **IdentityUser<Guid>**

- This ensures that ASP.NET Core Identity handles user authentication
- Uses GUID instead of an integer (int) for user IDs

Constructor: Generates a New GUID for Id

```
public ApplicationUser()  
{  
    this.Id = Guid.NewGuid();  
}
```

Navigation Properties

- **ApplicationUserMovies** - Represents the many-to-many relationship between users and movies
- **Tickets** - a collection of type **Ticket**

Cinema

- **Id** – Guid, Primary Key
- **Name** – string (required)
- **Location** – string (required)
- **IsDeleted** – bool
- **CinemaMovies** - a collection of type **CinemaMovie**
- **Tickets** - a collection of type **Ticket**

Movie

- **Id** – Guid, Primary Key
- **Title** – string (required)
- **Genre** – string (required)
- **ReleaseDate** – DateTime (required)
- **Director** – string (required)
- **Duration** – int (required, represents movie length in minutes)
- **Description** – string (required)
- **ImageUrl** – string?
- **IsDeleted** – bool
- **MovieCinemas** – a collection of type **CinemaMovie**
- **MovieApplicationUsers** – a collection of type **ApplicationUserMovie**
- **Tickets** – a collection of type **Ticket** (one-to-many relationship)

CinemaMovie

- **MovieId** – Guid, Foreign Key (References Movie)
- **Movie** – Movie (Navigation Property)

- **CinemaId** – **Guid, Foreign Key** (References Cinema)
- **Cinema** – **Cinema** (Navigation Property)
- **AvailableTickets** – **int**, Tracks remaining tickets for the movie in a specific cinema
- **IsDeleted** – **bool**
- **Showtimes** – **string** (formatted as "00000", stored as **varchar(5)**)

ApplicationUserMovie

- **ApplicationUserId** – **Guid, Foreign Key** (References ApplicationUser)
- **ApplicationUser** – a navigation property of type **ApplicationUser**
- **MovieId** – **Guid, Foreign Key** (References Movie)
- **Movie** – a navigation property of type **Movie**

Ticket

- **Id** – **Guid, Primary Key**
- **Price** – **decimal** (required)
- **CinemaId** – **Guid, Foreign Key** (References Cinema)
- **Cinema** – **Cinema** (Navigation Property)
- **MovieId** – **Guid, Foreign Key** (References Movie)
- **Movie** – a navigation property of type **Movie**
- **ApplicationUserId** – **Guid, Foreign Key** (References ApplicationUser)
- **ApplicationUser** – a navigation property of type **ApplicationUser**

Creating a ValidationConstants Class for Centralized Validations

A separate class to store validation rules could be created (option). This will help **keep the code clean, maintainable, and reusable**:

- **Avoid Hardcoded Values** – Keeps validation rules in a single place instead of spreading them across models
- **Improves Readability** – Models are cleaner and easier to understand
- **Easy Updates** – If a validation rule changes, update it in one place instead of multiple files

7. Creating CinemaAppDbContext

Now that we have created all the **models**, we need to set up the **database context**, which will serve as the main bridge between our application and the database. The **database context** is responsible for managing the entities and database operations using **Entity Framework Core (EF Core)**.

Important Note for Students: Remove the Legacy Data Folder

When setting up the **Entity Framework Core (EF Core)** project, **students must delete the legacy Data folder** that is automatically created when an ASP.NET project includes Identity by default.

Why is This Important?

Prevents Duplicate DbContext Issues

- The default Identity setup **creates an extra ApplicationDbContext** inside the Data folder.
- If not removed, EF Core **detects multiple DbContext classes**, leading to confusion when applying migrations.

Create the CinemaAppDbContext Class

- Inside **CinemaApp.Data**, create a new file called **CinemaAppDbContext.cs**
 - Extending **IdentityDbContext** for Authentication

```
0 references
public class CinemaAppDbContext : IdentityDbContext<ApplicationUser, IdentityRole<Guid>, Guid>
{
    0 references
    public CinemaAppDbContext(DbContextOptions<CinemaAppDbContext> options)
        : base(options)
    {
    }
}
```

- Extends **IdentityDbContext** to include **authentication support**
- Uses **GUID as the primary key** for users and roles
- Receives **database connection settings** via **dependency injection**
- Passes options to the **base constructor (IdentityDbContext)**

Defining Database Tables

```
0 references
public virtual DbSet<Movie> Movies { get; set; } = null!;
```

```
0 references
public virtual DbSet<Cinema> Cinemas { get; set; } = null!;
```

```
0 references
public virtual DbSet<CinemaMovie> CinemasMovies { get; set; } = null!;
```

```
0 references
public virtual DbSet<ApplicationUserMovie> UsersMovies { get; set; } = null!;
```

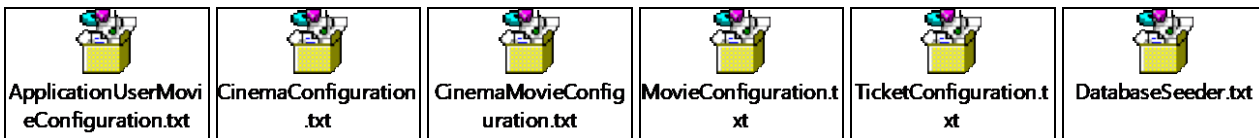
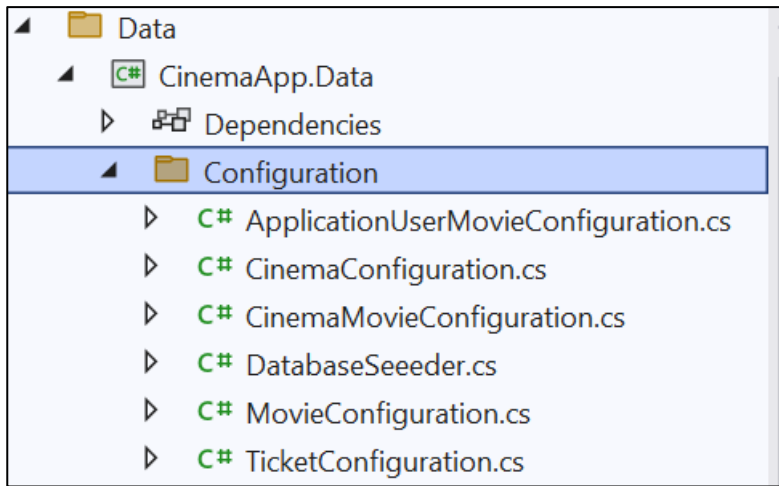
```
0 references
public virtual DbSet<Ticket> Tickets { get; set; } = null!;
```

8. Configuring Entities Using Fluent API

In Entity Framework Core, we can configure entity relationships and constraints using **Data Annotations** or **Fluent API**. While **Data Annotations** work well for simple cases, using **Fluent API in separate configuration classes** provides **better control, flexibility, and maintainability**.

Using Configuration Classes

- Inside **CinemaApp.Data**, create a **new folder** named **Configuration**
- Create a Configuration Class for Each Model



9. Understanding OnModelCreating in CinemaAppDbContext

The **OnModelCreating** method in **Entity Framework Core (EF Core)** is responsible for configuring the **database schema, relationships, and entity constraints** before the database is created.

Why Do We Need OnModelCreating?

Defines Entity Relationships – Ensures **correct table relationships (One-to-Many, Many-to-Many)**.

✓ **Applies Fluent API Configurations** – Loads **entity configurations from the Configuration folder**.

✓ **Applies Identity Settings** – Calls `base.OnModelCreating(modelBuilder)` to apply **ASP.NET Core Identity configurations**.

✓ **Allows Data Seeding** – Adds **default data** to tables during database creation.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.ApplyConfigurationsFromAssembly(Assembly.GetExecutingAssembly());
}
```

`base.OnModelCreating(modelBuilder);`

- Calls the **base class implementation** (from `IdentityDbContext`)
- This ensures that **Identity tables** (`AspNetUsers`, `AspNetRoles`, etc.) are created properly

`modelBuilder.ApplyConfigurationsFromAssembly(Assembly.GetExecutingAssembly());`

- **Automatically applies all entity configurations** from the Configuration folder
- Instead of defining relationships in `DbContext`, we keep them **separate for better maintainability**

10. Updated Program.cs for This Stage

- Database Connection Configuration

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
string connectionString = builder.Configuration.GetConnectionString("SQLServer");

builder.Services
    .AddDbContext<CinemaDbContext>(options =>
    {
        options.UseSqlServer(connectionString);
    });
```

- Identity Setup

```
builder.Services
    .AddIdentity<ApplicationUser, IdentityRole<Guid>>(cfg =>
    {
        ConfigureIdentity(builder, cfg);
    })
    .AddEntityFrameworkStores<CinemaDbContext>()
    .AddRoles<IdentityRole<Guid>>()
    .AddSignInManager<SignInManager<ApplicationUser>>()
    .AddUserManager<UserManager<ApplicationUser>>();
```

- Middleware Setup

```
app.UseAuthentication();
app.UseAuthorization();
```

- Routing Setup

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.MapRazorPages();
```

Configure appsettings.json to include the database connection string

11. Add-Migration & Update-Database

12. Verify the Database in SQL Server Management Studio (SSMS)

<